AI TOPICS COURSE · TEAM PROJECT GUIDE · DOCUMENT 0 OF 5

Regulatory & Support Copilot

A general product any organization can customize. This is the master guide for the whole project. Read it first, then read your own part guide.

Part	Title	Owner	Guide
1	Templates, Mock Data and Connectors	Zain Allateef	01-Part1-Templates-Data-and-Connectors.pdf
2	Knowledge Base and AI Foundation		02-Part2-Knowledge-Base-and-AI-Foundation.pdf
3	Change Watch Agents		03-Part3-Change-Watch-Agents.pdf
4	Request Desk Agent and Orchestrator		04-Part4-Request-Desk-and-Orchestrator.pdf
5	Website, Safety and Evaluation		05-Part5-Website-Safety-and-Evaluation.pdf

Contents

1. How to use these documents
2. The product in one page
3. What the finished prototype does
4. System architecture
5. The five parts at a glance
6. Technology stack
7. Repository structure
8. Shared contracts
9. Data policy: templates and mock data
10. Environment setup (every member)
11. GitHub workflow
12. Building your part with your own AI tool
13. Timeline
14. Integration and definition of done
15. Knowing each other's work
16. Final deliverables
17. Glossary

1. How to use these documents

There are six guides. This one describes the whole product and the rules everyone follows. The other five each describe one part, in enough detail for its owner to build it step by step.

- **Everyone reads this whole guide.** In the final demo you will be asked about all parts, not only yours.
- **Read your part guide twice**, then skim the other four so you know what your teammates build and what they expect from you.
- **Use your `PROMPTS.md`.** Each part folder has a `PROMPTS.md` with five ready prompts for your AI coding tool. Start with Prompt 1.

- **The shared contracts in Section 8 are the law.** If your code follows them, the five parts fit together. To change a contract, open a pull request and get approval from the whole team (Section 11).
- **When something is unclear**, open a GitHub Issue with the label `question` instead of guessing.

> **Guide's note:** The biggest risk in a 5-person project is five pieces that don't connect. The second biggest risk in a *general* product is code that secretly only works for one organization. Every rule in this guide fights one of these two risks.

2. The product in one page

Where the idea came from. The project started as a copilot for payments software companies like ProgressSoft (Amman, Jordan), which build payment and clearing systems for central and commercial banks. Our doctor advised us to turn it into a product any organization can customize and buy. So we build a general product, and a payments software company stays our **main example**.

The two problems every organization has.

1. **New rules keep arriving.** A payments software company receives a new central bank circular (for example a new ISO 20022 field rule); a university gets a new accreditation rule. Someone must read it, find which product modules or study programs and which client banks or student groups it affects, and plan the work. This takes days and depends on a few experts.
2. **Requests keep arriving.** Client banks ask "Why was payment 7731 rejected?"; students ask "Why can't I register for SE301?". Someone must dig through records to find the reason and reply. Replies are slow.

Our product. One core engine with AI agents, customized by a **template** per organization:

- **Change Watch (Workflow A):** Change Watcher summarizes a new rule → Impact Analyst finds affected units and stakeholder groups → Action Planner drafts action items → a human approves, edits or rejects.
- **Request Desk (Workflow B):** Request Investigator reads a request → looks up the person's records → finds the reason → drafts a reply → a human approves it before it is "sent".

A template is a set of files: the organization's name, its words (labels), its units and stakeholder groups, its documents, which record connector to use, and prompt hints. **The code never changes per organization.**

What we build. A working **prototype website** with two templates: **Payments** (Cedar Payment Systems, a fictional payments software company like ProgressSoft, the **main demo**) and **University** (Northfield University, the second demo, which proves the product is general). Every agent step shows live, every answer shows its sources, and everything is audited. Plus an evaluation report per template. Other organizations (for example a hospital, a telecom or an online store) are only mentioned as possible future customers.

What we do not build. No real company, bank or university data, no real payments, no real student records, no real connections to outside systems, no production login (the reviewer picks a name).

3. What the finished prototype does

The final presentation is a **live demo of this website** to the doctor and the students. Build toward these three demos from week 1. If a feature does not serve them, it is optional.

Demo A: Payments, a new central bank circular arrives (Change Watch)

1. The active template is **Payments** (Cedar Payment Systems). Open **New Change**, choose `CHG-P-002` ("Structured creditor address mandatory in credit transfers") and click **Analyze**.

2. The **Task page** shows the three agent steps running one after another: waiting, running, done or failed, with times.
3. When done, it shows:
    - the **summary**: each requirement (for example "payments without a structured creditor address must be rejected with reason code BE04"), its effective date, and the exact sentence it came from;
    - the **impact report**: affected product modules (ACH, IPS), affected client banks (BANK-A, BANK-B, BANK-E in Country X), severity, and the product documents used as evidence;
    - the **draft action items** (change requests) with owner role, due date and checks.
4. The reviewer (a business analyst) picks their name and clicks **Approve**, **Edit** or **Reject** on each action item.
5. Approved items appear on the **Action Items** page. The **Audit Log** shows every step and decision.

Demo B: Payments, a bank support ticket (Request Desk)

1. Open **Requests**, choose `P-REQ-007` from client bank BANK-C ("Why was payment 7731 rejected?") and click **Investigate**.
2. The Task page shows the steps: find the payment reference, search the transaction records, search the documents, draft the reply.
3. The result shows the reason code (`AC04`, closed account), the transaction records used as evidence, the sources (`reason_codes.md`), a confidence level, and a draft reply signed "Support Team, Cedar Payment Systems".
4. The reviewer (a support engineer) approves the reply. The request status changes to "Answered".

Demo C: switch to University (proof that the product is general)

1. Use the **template switcher** in the navigation bar to choose **University**. Every label and the organization name change (product module → study program, client bank → student group, transaction record → registration record).
2. Open `U-REQ-007` from student S-2291 ("Why can't I register for SE301?") and click **Investigate**. The same agent finds `BLOCKED · PREREQ_MISSING` (SE201 not passed) and drafts a reply signed by the Registrar's Office.
3. Open the **Setup** page: it shows the university template's organization, labels, units, stakeholder groups, connector and document count. Say: "Same code, different template."

Extra pages

- **Knowledge Search:** type a question and see which document chunks are found for the active template. This explains RAG.
- **Evaluation:** the latest accuracy numbers for each template.

4. System architecture



How a request flows (Workflow A): the website calls `start_change_task()` → the orchestrator creates a task for the active template and calls the three Change Watch agents in order → each agent loads the template, fills its prompt with the template's labels (`prompting.render`), uses `search()` for evidence, `llm.complete_json()` to think, and Part 1 tools for facts → the orchestrator saves each output and writes audit events → the website polls the task every 2 seconds → the human decision is saved and audited.

How templates make it general: every Part 1 and Part 2 function takes an optional `template_id` (none = the active template). The record lookup goes through a **connector** chosen in `template.yaml`: `transaction_lookup` for the payments company, `registration_lookup` for the university. Agents never contain words like "bank", "payment" or "student"; those come from the template's labels.

5. The five parts at a glance

Part	Folder	Delivers	Needs from	Gives to
1 Templates, Mock Data and Connectors	`part1_templates/`	Two templates with all mock data, record connectors, read-only tools, evaluation answer sets	—	Everyone
2 Knowledge Base and AI Foundation	`part2_knowledge/`	Ingestion and `search()` per template, `llm.py`, `config.py`, `prompting.py`	Part 1	Parts 3, 4, 5
3 Change Watch Agents	`part3_change_watch/`	Change Watcher, Impact Analyst, Action Planner	Parts 1, 2	Part 4 (orchestrator)
4 Request Desk Agent and Orchestrator	`part4_request_desk/`	Request Investigator, task runner, task store	Parts 1, 2, 3, 5 (audit)	Part 5 (website)

Part	Folder	Delivers	Needs from	Gives to
5 Website, Safety and Evaluation	`part5_web/`	Website with template switcher and Setup page, masking, audit log, evaluation	Parts 1, 2, 4	The demo

Review pairs (who reviews whose pull requests): Part 1 → reviewed by Part 2 · Part 2 → by Part 3 · Part 3 → by Part 4 · Part 4 → by Part 5 · Part 5 → by Part 1. Changes to `contracts/` need approval from **all** members.

6. Technology stack

These choices are fixed so the parts fit together. Don't swap a library without a team decision recorded in an Issue.

Area	Choice	Why
Language	Python 3.11 or newer	One language for everything; every AI coding tool knows it well
Data models	Pydantic v2	Shared contracts with automatic validation
Templates	YAML (`template.yaml`, read with `pyyaml`) + JSON / Markdown data files	Easy for people to read and edit; no code per organization
Web	FastAPI + Jinja2 HTML templates + plain JavaScript	Simple, no front-end build step
Database	SQLite (one file, `runtime/app.db`)	No server to install
Vector database	Chroma (local, `runtime/chroma/`), one collection per template (`kb_payments`, `kb_university`)	Free, offline, keeps organizations separate
Embeddings	Chroma's default model (all-MiniLM-L6-v2)	Free and local; about 80 MB download on first use
LLM	One wrapper `part2_knowledge/common/llm.py` with providers `fake`, `ollama`, `anthropic`, `openai`	Develop free with `fake` or `ollama`; demo can use a stronger model
Agents	Plain Python functions (no agent framework)	Easy to read, test and explain
Tests	pytest	Standard and simple
Collaboration	GitHub: Issues, Projects board, pull requests	One place for code, tasks and reviews

About the LLM providers. `fake` returns the example object from the contract, so code runs with no model (tests, early integration). `ollama` runs a free local model such as `qwen2.5:7b` or `llama3.1:8b` (about 8 GB RAM). `anthropic` or `openai` need a paid API key; a small model is enough. The team picks one provider for the demo and puts it in `.env`.

> **Note on the word "template":** in this project a *template* means an organization's configuration (`part1_templates/templates/payments/`). The website's HTML files in `part5_web/templates/` are Jinja2 HTML templates, a different thing. Say "HTML template" when you mean those.

7. Repository structure

The Git repository is `AI-Topics-Course-Project`. The product lives in `projects/Regulatory-Support-Copilot/`. **Run every command from that folder.** Each part has one folder, edited only by its owner. Only `contracts/` and `tests/integration/` are shared. Empty folders hold a `.gitkeep`, and every Python folder has an `__init__.py` so parts can import each other (for example `from part1_templates.tools.records import search_records`).

```
AI-Topics-Course-Project/
├── .github/
│   ├── CODEOWNERS                maps each part folder to its owner
│   └── pull_request_template.md
└── projects/Regulatory-Support-Copilot/       ← run every command from here
    ├── README.md                 how to install, run and demo (Part 5 keeps it updated)
    ├── requirements.txt          all Python packages (add yours by pull request)
    ├── .env.example              every setting, with no secret values
    ├── .gitignore, .gitattributes, pytest.ini
    ├── contracts/                SHARED: schemas.py + CONTRACTS.md (all approve changes)
    │
    ├── part1_templates/          PART 1 – Templates, Mock Data and Connectors
    │   ├── templates/
    │   │   ├── payments/         MAIN: template.yaml, units.json, stakeholders.json, documents/,
    │   │   │                     changes/, records/, requests/, history/, eval/, generators/
    │   │   └── university/       SECOND: the same layout, smaller data set
    │   ├── connectors/           base.py, transaction_lookup.py, registration_lookup.py
    │   ├── tools/                templates.py, changes.py, directory.py, requests.py, records.py,
action_list.py
    │   └── tests/
    ├── part2_knowledge/          PART 2 – Knowledge Base and AI Foundation
    │   ├── knowledge_base/       ingest.py, search.py
    │   ├── common/               config.py, llm.py, prompting.py
    │   └── tests/
    ├── part3_change_watch/       PART 3 – Change Watch Agents
    │   ├── watcher.py, impact.py, action_planner.py, cli.py
    │   ├── prompts/
    │   └── tests/
    ├── part4_request_desk/       PART 4 – Request Desk Agent and Orchestrator
    │   ├── investigator.py
    │   ├── prompts/
    │   ├── orchestrator/         orchestrator.py, store.py, run.py
    │   └── tests/
    ├── part5_web/                PART 5 – Website, Safety and Evaluation
    │   ├── app.py, templates/ (HTML), static/
    │   ├── safety/               masking.py, audit.py, guidelines.md
    │   ├── evaluation/           run_eval.py
    │   └── tests/
    │
    ├── tests/integration/        SHARED: end-to-end tests (Parts 4 and 5)
    ├── docs/                     these guides (PDF), docs/source/*.md, overview, proposal
    ├── presentation/             animated explainer
    └── runtime/                  created at run time (database, Chroma, state); never committed
```

Main commands (from `projects/Regulatory-Support-Copilot/`):

```
python -m part2_knowledge.knowledge_base.ingest                # build the search index (all templates)
uvicorn part5_web.app:app --reload                             # website at http://127.0.0.1:8000
python -m part3_change_watch.cli CHG-P-002                     # run Change Watch without the website
python -m part4_request_desk.orchestrator.run --request P-REQ-007   # also --change CHG-P-002, --list
python -m part5_web.evaluation.run_eval --template payments    # or university, or all
pytest                                                         # all tests
```

Each part folder has its own `README.md`, `AI_LOG.md` and `PROMPTS.md`.

8. Shared contracts

Contracts are the data shapes and function signatures that connect the parts. They live in `contracts/schemas.py` as Pydantic models. Each model must include a `Field(description=...)` on every field and one realistic example (in `model_config = {"json_schema_extra": {"example": {...}}}`), because the `fake` LLM provider returns that example and the website uses it for previews. **Examples use payments data** (the main template), so everyone shares one mental picture.

Contract Day (end of week 1): each owner writes the models for their part in one pull request, and everyone reviews all of them together in a 1-hour meeting. After that, changes need everyone's approval.

> **Rule for a general product:** no contract field may contain an organization-specific word (no `bank_id`, no `payment_ref`, no `student_id`). Use the general words below; the template gives them labels.

8.1 Data models

Model	Written by	Key fields
`TemplateConfig`	Part 1	`id`, `organization`, `languages`, `labels` (dict), `connector`, `item_pattern`, `reason_codes_doc`, `prompt_hints`, `action_item_fields`, `reviewers`
`Unit`	Part 1	`unit_id`, `name`, `type`, `description`
`Stakeholder`	Part 1	`stakeholder_id`, `name`, `type`, `units` (list of unit ids), `attributes` (dict)
`ChangeDocument`	Part 1	`change_id`, `template_id`, `title`, `issuer`, `scope`, `published` (date), `effective` (date or null), `text`
`Record`	Part 1	`record_id`, `template_id`, `timestamp`, `person_id`, `unit_id`, `item`, `status`, `reason_code` (or null), `detail`
`Request`	Part 1	`request_id`, `template_id`, `person_id`, `created_at`, `subject`, `body`, `priority`, `status` (open / answered), `item` (or null)
`SearchResult`	Part 2	`text`, `source` (file path), `doc_type`, `location` (section title), `score` (0–1), `metadata` (dict)
`Requirement`	Part 3	`requirement_id`, `description`, `category`, `mandatory`, `source_quote`, `source_location`
`ChangeSummary`	Part 3	`change_id`, `template_id`, `title`, `scope`, `effective_date`, `summary_text`, `requirements` (list of `Requirement`)
`Impact`	Part 3	`requirement_id`, `unit_id`, `stakeholders` (list of ids), `severity` (high / medium / low), `reason`, `sources` (list of `SearchResult`)
`ImpactReport`	Part 3	`change_id`, `impacts` (list of `Impact`), `unmatched_requirements` (list of ids)
`Check`	Part 3	`check_id`, `steps` (list), `expected`
`ActionItem`	Part 3	`action_id`, `change_id`, `title`, `description`, `unit_id`, `stakeholders`, `owner_role`, `due_before` (date or null), `checks` (list of `Check`), `sources`, `status` (pending_review / approved / rejected)
`Investigation`	Part 4	`request_id`, `item`, `reason_code`, `root_cause`, `evidence` (list of `Record`), `sources` (list of `SearchResult`), `draft_reply`, `confidence` (high / medium / low), `escalate` (bool)
`TaskStep`	Part 4	`name`, `status` (waiting / running / done / failed), `started_at`, `finished_at`, `error`, `message`

Model	Written by	Key fields
`Task`	Part 4	`task_id`, `template_id`, `type` (change / request), `status` (running / pending_review / approved / rejected / failed), `input_ref`, `steps`, `outputs` (dict), `created_at`
`ReviewDecision`	Part 5	`task_id`, `item_id`, `decision` (approve / edit / reject), `edited_content` (or null), `reviewer`, `comment`, `timestamp`
`AuditEvent`	Part 5	`timestamp`, `template_id`, `actor` (agent name or reviewer), `action`, `task_id`, `details` (dict)

Example of one model in JSON form, to show the level of detail expected:


```
{
  "request_id": "P-REQ-007",
  "item": "7731",
  "reason_code": "AC04",
  "root_cause": "Payment 7731 was rejected because the receiving account at BANK-C is closed.",
  "evidence": [{"record_id": "PR-00388", "template_id": "payments", "timestamp": "2026-03-02T09:12:03",
                "person_id": "BANK-C", "unit_id": "ACH", "item": "7731", "status": "rejected",
                "reason_code": "AC04", "detail": "pacs.008 rejected: creditor account closed"}],
  "sources": [{"text": "AC04 – Closed account number. The creditor account is closed ...",
               "source": "part1_templates/templates/payments/documents/reason_codes.md",
               "doc_type": "reason_codes", "location": "AC04", "score": 0.84, "metadata": {"reason_code":
"AC04"}}],
  "draft_reply": "Dear BANK-C team, payment 7731 was rejected because the receiving account is closed ...",
  "confidence": "high",
  "escalate": false
}
```


8.2 Function interfaces

Each part exposes these functions. Other parts call **only** these, never internal helpers. Every Part 1 and Part 2 function takes an optional `template_id=None`; `None` means the active template from `.env` (`ACTIVE_TEMPLATE`).

Owner	Function	Returns
Part 1	`part1_templates.tools.templates.list_templates()` / `load_template(template_id=None)`	`list[TemplateConfig]` / `TemplateConfig`
Part 1	`part1_templates.tools.changes.list_changes()` / `get_change(change_id)` / `parse_change(text)`	`ChangeDocument` objects
Part 1	`part1_templates.tools.directory.list_units()` / `list_stakeholders()` / `stakeholders_for_unit(unit_id, scope=None)`	lists of `Unit` / `Stakeholder`
Part 1	`part1_templates.tools.requests.list_requests()` / `get_request(id)` / `set_request_status(id, status)`	`Request` objects
Part 1	`part1_templates.tools.records.search_records(person_id=None, unit_id=None, item=None, status=None, since=None, until=None)`	`list[Record]` (calls the template's connector)
Part 1	`part1_templates.tools.action_list.create_action_item(item)` / `list_action_items()`	id / list (state in `runtime/<template>/state.json`)
Part 2	`part2_knowledge.knowledge_base.search.search(query, k=5, filters=None, template_id=None)`	`list[SearchResult]`

Owner	Function	Returns
Part 2	`part2_knowledge.knowledge_base.ingest.ingest_all(template_id=None)` (None = all templates)	stats dict
Part 2	`part2_knowledge.common.llm.complete_text(system, user)` / `complete_json(system, user, schema)`	`str` / instance of `schema`
Part 2	`part2_knowledge.common.prompting.render(prompt_text, template)`	prompt with `{organization}`, `{unit_label}`, `{prompt_hints}` ... filled in
Part 3	`part3_change_watch.watcher.summarize(doc)`	`ChangeSummary`
Part 3	`part3_change_watch.impact.analyze(summary)`	`ImpactReport`
Part 3	`part3_change_watch.action_planner.plan(summary, report)`	`list[ActionItem]`
Part 4	`part4_request_desk.investigator.investigate(request)`	`Investigation`
Part 4	`part4_request_desk.orchestrator.start_change_task(doc)` / `start_request_task(request)`	`task_id` (runs in background)
Part 4	`part4_request_desk.orchestrator.get_task(id)` / `list_tasks(template_id=None, type=None, status=None)` / `set_task_status(id, status)` / `update_output(id, key, value)` / `retry_task(id)`	`Task` objects
Part 4	`part4_request_desk.orchestrator.run_change_sync(doc)` / `run_request_sync(request)` (no background thread, for evaluation)	finished `Task`
Part 5	`part5_web.safety.masking.mask(text)`	masked `str`
Part 5	`part5_web.safety.audit.log_event(actor, action, task_id, details, template_id=None)` / `list_events(task_id=None)`	`None` / `list[AuditEvent]`

Stubs rule. Until a function is ready, its owner merges a **stub**: the real signature that returns the contract example (or, for `mask`, the text unchanged). Everyone else can then build and test from week 2 without waiting.

9. Data policy: templates and mock data

We have no real data from ProgressSoft, any bank or any university, and we must never use any. Part 1 creates two complete fictional organizations:

	Payments template (main)	University template (second)
Organization	Cedar Payment Systems (a payments software company like ProgressSoft)	Northfield University
Units	5 product modules: CHQ, ACH, IPS, WAL, RPT	6: CS-BSc, SE-BSc, BUS-BBA, NUR-BSc, REG-OFF, INTL-OFF
Stakeholder groups	5 client banks: BANK-A ... BANK-E	6 student and staff groups
Change documents	5 central bank circulars (CHG-P-001 ... 005)	3 (CHG-U-001 ... 003)
Records	about 500 transaction records	about 250 registration records

	Payments template (main)	University template (second)
Reason codes	12 real ISO 20022 codes (AC01, AC04, AM04, BE04 ...)	PREREQ_MISSING, SECTION_FULL, FEES_UNPAID ...
Requests	15 bank support tickets (P-REQ-001 ... 015)	10 student requests (U-REQ-001 ... 010)
Evaluation	full answer sets	small answer sets

Rules:

- **Fictional names only.** No real company, bank, central bank, university, student or staff member. ProgressSoft is named only as the real company that inspired the main template; its data is never used. Organizations and regulators are invented ("Cedar Payment Systems", "Central Bank of Country X", "Higher Education Accreditation Commission").
- **Real public standards are allowed:** ISO 20022 message names (pacs.008, pacs.002) and reason codes (AC01, AC04, BE04 ...) are public and make the payments template realistic.
- **AI-generated text is allowed and expected** for documents, change documents and requests. A human must check every generated file for consistency (Part 1 guide).
- **Records are generated by a script with a fixed random seed** (`seed 42`), so every member gets identical data.
- **Mark it.** Every data file starts with `MOCK DATA – fictional, for a student prototype` (in JSON files, a `"_note"` field).
- **Fake personal data for safety tests** must be clearly invented: IBAN-shaped numbers (`JO71 CBJO 0010 0000 0000 0131 0003 02`), long account numbers, card numbers, 10-digit national IDs, phone numbers like `+962 7X XXX XXXX`, and emails.

10. Environment setup (every member)

Do this in week 1, before writing any code. Ask in the team chat if a step fails.

1. **Install the tools:** Git, Python 3.11+, VS Code (or your editor) and your AI coding tool. Optional: Ollama (free local LLM) from ollama.com, then `ollama pull qwen2.5:7b`.
2. **Create a GitHub account** if needed and send your username to the team lead.
3. **Tell Git who you are** (one time):

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

4. **Accept the invitation** to the repository, then clone it and enter the project folder:

```
git clone https://github.com/Nadaalnajjar2015/AI-Topics-Course-Project.git
cd AI-Topics-Course-Project/projects/Regulatory-Support-Copilot
```

5. **Create a virtual environment and install packages:**

```
python -m venv .venv
.venv\Scripts\activate          (Windows)
source .venv/bin/activate       (macOS / Linux)
pip install -r requirements.txt
```

6. **Create your settings file:** copy `.env.example` to `.env`. Start with `LLM_PROVIDER=fake`. Never commit `.env`.

```
ACTIVE_TEMPLATE=payments     # payments | university
TEMPLATES_DIR=part1_templates/templates
LLM_PROVIDER=fake            # fake | ollama | anthropic | openai
LLM_MODEL=qwen2.5:7b
OLLAMA_BASE_URL=http://localhost:11434
ANTHROPIC_API_KEY=
OPENAI_API_KEY=
CHROMA_DIR=runtime/chroma
APP_DB=runtime/app.db
```

7. **Check that it runs** (once the skeletons are merged): `pytest` passes and `uvicorn part5_web.app:app --reload` opens the website at http://127.0.0.1:8000.

`requirements.txt` contains: `fastapi`, `uvicorn[standard]`, `jinja2`, `python-multipart`, `pydantic>=2`, `pyyaml`, `python-dotenv`, `chromadb`, `httpx`, `anthropic`, `openai`, `lxml`, `pytest`.

11. GitHub workflow

11.1 One-time setup (team lead)

1. The repository `AI-Topics-Course-Project` already exists. **Settings → Collaborators:** invite the four other members with *Write* access.
2. **Branch protection** for `main` (Settings → Branches): require a pull request, 1 approval and Code Owner review. On a free account this needs a public repository or GitHub Pro (free with the GitHub Student Developer Pack). Until then, the rule "never push to main" is a team agreement.
3. `.github/CODEOWNERS` lives at the **repository root** and uses full paths. Replace the remaining usernames:

```
/projects/Regulatory-Support-Copilot/part1_templates/     @zaynallateef
/projects/Regulatory-Support-Copilot/part2_knowledge/     @member2
/projects/Regulatory-Support-Copilot/part3_change_watch/  @member3
/projects/Regulatory-Support-Copilot/part4_request_desk/  @member4
/projects/Regulatory-Support-Copilot/part5_web/           @member5
/projects/Regulatory-Support-Copilot/tests/integration/   @member4 @member5
/projects/Regulatory-Support-Copilot/contracts/           @zaynallateef @member2 @member3 @member4 @member5
```

4. `.github/pull_request_template.md` has the headings: *What this PR does · Which part · How I tested it · Screenshots (for website changes) · Contract changes? (yes/no)*.
5. **Projects → New project → Board** with columns *To do*, *In progress*, *In review*, *Done*. Labels: `part-1` … `part-5`, `bug`, `question`, `contract`, `integration`, `template`.
6. Copy the issue list from each part guide into Issues and assign them.

11.2 Daily routine (everyone)

```
cd AI-Topics-Course-Project/projects/Regulatory-Support-Copilot
git checkout main
git pull                                  # get everyone's latest work
git checkout -b part3/watcher             # one branch per task
... write code + tests ...
pytest part3_change_watch/tests           # your tests must pass
git add part3_change_watch
git commit -m "feat(part3): add change watcher with source-quote check"
git push -u origin part3/watcher
```

Then on GitHub: **open a pull request** into `main`, fill the template, link the issue (`Closes #12`), and request your reviewer. After approval, **Squash and merge** and delete the branch.

11.3 Rules

- **Branch names:** `partN/short-description`, for example `part1/payments-template`, `part5/setup-page`.
- **Commit messages:** `type(partN): what changed`, where type is `feat`, `fix`, `test`, `docs` or `refactor`.
- **Small pull requests**, under about 400 changed lines, at least every 3–4 days.
- **Only edit your own part folder.** Need a change elsewhere? Open an Issue and assign the owner.
- **Test with both templates.** A pull request that works for the payments template but breaks the university is not done.
- **Reviewing is part of your grade.** At least one real comment or question per review.
- **Never commit:** `.env`, API keys, `.venv/`, `runtime/`, files over 5 MB.
- **Pull `main` into your branch** (`git pull origin main`) before opening a PR.

11.4 Fixing a merge conflict

1. Run `git pull origin main` on your branch. Git lists the conflicted files.
2. In each file, keep the right lines between `<<<<<<<`, `=======` and `>>>>>>>`, then delete the markers.
3. Run the tests, then `git add <file>`, `git commit`, `git push`.
4. If the conflict is in someone else's code, call them before choosing.

12. Building your part with your own AI tool

Each member may use any AI coding tool (Claude Code, Cursor, Copilot, ChatGPT ...). These rules keep five tools producing one coherent product.

12.1 Rules

1. **Use your `PROMPTS.md`.** It has five prompts in order. Prompt 1 makes the tool study the whole repository first.
2. **Open the tool inside the repository** so it can read the guides in `docs/source/*.md` (Markdown versions of these PDFs).
3. **Work in small steps:** one function or one page at a time, with tests.
4. **You must understand every line you merge.** Anyone may be asked about any line in the demo.
5. **The tool must not change other parts' folders or the contracts.** If it suggests that, stop and open an Issue.
6. **Never paste secrets** into an AI tool or into code.
7. **No organization-specific code.** If the tool writes "bank", "payment" or "student" in agent code, move the word into the template.
8. **Keep `AI_LOG.md` in your folder:** the main prompts and what you changed by hand.

12.2 Starter prompt shape

```
You are helping me build Part N of a 5-person student project, the "Regulatory & Support
Copilot", a general product customized per organization by templates (payments, university).
Work in projects/Regulatory-Support-Copilot of the AI-Topics-Course-Project repository.
Read the guides in docs/source/, contracts/schemas.py and <my folder>/PROMPTS.md first.
Rules: Python 3.11, Pydantic v2, follow the contracts exactly, edit only <my folder>, no
organization-specific words in code, pytest tests for every function, work on a new branch.
Show me a plan first and wait for my approval. Then do task: <one task>.
```

13. Timeline

The plan assumes 8 weeks. If your deadline differs, keep the order and stretch or squeeze the weeks.

Week	Everyone	Milestone
1	Setup (Section 10), Contract Day, Part 1 writes both `template.yaml` files	**M1:** contracts and template format merged
2	Stubs for every interface; Part 1 delivers a payments data sample	Everyone can import everyone's functions
3	Core building with the `fake` LLM on the payments template	Each part has working code + tests
4	**First integration:** Demos A and B run on the website with `fake` LLM	**M2:** clickable end-to-end prototype
5	Real LLM; Part 1 delivers the university template; everyone tests with both templates	Demo C works
6	Quality week: bugs, errors, website polish, safety; feature freeze at the end	All three demos stable
7	Evaluation for both templates, final data fixes, report	**M3:** evaluation report ready
8	Rehearse the live demo twice, record a backup video, final report	**M4:** final demo

Weekly meeting (30 minutes): each member shows a 3-minute demo of what they merged, says what is next and names any blocker.

14. Integration and definition of done

14.1 Integration order

1. Part 1 first: both `template.yaml` files and the payments data; Part 2 ingests and searches them.
2. Parts 3 and 4 agents use `search()`, `prompting.render()` and the tools, tested alone from the command line.
3. Part 4's orchestrator connects the agents (`python -m part4_request_desk.orchestrator.run`).
4. Part 5's website calls the orchestrator; Demos A and B in the browser.
5. University template added; Demo C; safety and evaluation last.

Integration tests live in `tests/integration/` (Parts 4 and 5 together): one test per workflow **per template**, using the `fake` provider.

14.2 The product is done when

- [ ] Demos A, B and C (Section 3) work on a fresh clone by following README.md.
- [ ] Switching the template changes every label, the organization name and the data, with no code change.
- [ ] Every agent output shows its sources; nothing is "sent" or saved as an action item without approval.
- [ ] Personal data (IBANs, account and card numbers, national IDs, phones, emails) is masked before any LLM call (tested), and the prompt-injection tests P-REQ-015 and U-REQ-010 pass.
- [ ] The audit log records every agent step and human decision.
- [ ] `pytest` passes; each part has tests for its public functions; integration tests run for both templates.
- [ ] The evaluation report exists for both templates and is shown on the website.
- [ ] Each folder has a README and an `AI_LOG.md`.

15. Knowing each other's work

- **Review rotation** (Section 5) means you read another part's code every week.
- **Weekly 3-minute demos** show everyone's progress.
- **Each folder README** explains in one page what the part does, its inputs and outputs, and how to run it.

- **Week 7 swap session (1 hour):** each member explains a teammate's part; the owner corrects them.
- **Final demo:** each member presents their part, but questions may go to anyone.

16. Final deliverables

Deliverable	Where	Owner
Working prototype website with two templates	GitHub repository	All
README with install, run and demo steps	`README.md`	Part 5 (all contribute)
Evaluation report for both templates	`docs/evaluation-report.md` + Evaluation page	Part 5 (data from Part 1)
Final report (problem, product, design, results, lessons)	`docs/final-report.pdf`	All, one section each
Live demo (Demos A, B, C) + backup screen recording	presentation day	All
Animated explainer	`presentation/copilot-explainer.html`	—

Suggested final report outline: 1 Problem · 2 The product and templates · 3 Architecture · 4 One section per part (by its owner) · 5 Safety · 6 Evaluation per template · 7 How a new organization would adopt it · 8 Limitations and future work · 9 What we learned · Appendix: AI tool usage.

17. Glossary

Term	Meaning
Template	One organization's configuration: `template.yaml` plus its data files (for example `templates/payments/`)
Active template	The template the system is using now (`ACTIVE_TEMPLATE` in `.env`, or the website switcher)
Label	The organization's word for a general concept (unit → "Product module" or "Study program")
Unit	A part of the organization a rule can affect: a product module (ACH), a study program, an office
Stakeholder	A group affected by a rule: a client bank, a student group, a staff group
Person	Whoever sends requests and has records (client bank BANK-C, student S-2291)
Record	One entry in the organization's records (a payment transaction, a registration attempt)
Item	What a record is about (a payment reference like 7731, a course code like SE301)
Payment reference	The number that identifies one payment in the transaction records (for example 7731)
Connector	A small plug-in that reads one kind of record (`transaction_lookup`, `registration_lookup`)
Change document	A new rule, central bank circular or policy that the Change Watch workflow analyzes
ISO 20022	The international standard for payment messages (pacs.008 = customer credit transfer, pacs.002 = payment status)
Requirement	One concrete rule extracted from a change document, with its exact quote
Action item	A planned piece of work caused by a rule change, with owner, due date and checks
Request	A question or complaint from a person, handled by the Request Desk workflow

Term	Meaning
Reason code	A short code explaining why a record failed (ISO 20022 AC04 = closed account; PREREQ_MISSING for the university)
Agent	A Python function that uses an LLM plus tools to do one job and return a structured result
Orchestrator	The code that runs the agents in order and keeps each task's state
LLM	Large Language Model, the "brain" that reads and writes text
RAG	Retrieval-Augmented Generation: search our documents first, then answer from them
Embedding / vector database	Numbers that represent meaning; Chroma finds texts with similar meaning
Human-in-the-loop	A person must approve before an AI result is used
Contract / stub	The agreed data shape or signature / a placeholder that returns example data
Mock data	Realistic but invented data used instead of real organization data